

MEMORIAL UNIVERSITY OF NEWFOUNDLAND
Faculty of Engineering and Applied Science
Master of Applied Science (MAsc) in Software Engineering

Performance vs. Cost Efficiency in Kubernetes-Based Microservice Autoscaling

COMP 6910 — Services Computing,
Semantic Web and Cloud Computing

Winter 2026

Team Members

Name	Student ID
Afiya Farjana	202486149
Jannatul Ferdous	202487105
Rumnaz Khan	202487843

Course Information

Role	Name
Instructor	Dr. Kaiyang Liu
Semester	Winter 2026
Program	MAsc Software Engineering

Submitted: April 10, 2026

Table of Contents

Abstract	2
1 Introduction	2
2 Background and Related Work	3
2.1 Kubernetes HPA	3
2.2 Alpha-Beta Network Model	3
2.3 Related Work	3
3 Methodology	4
3.1 Workload and Application	4
3.2 Experimental Environments	4
3.3 Experiment Design	4
3.4 Cost Model	4
3.5 Network Latency Characterisation	5
3.6 HPA Validation	5
4 Results	5
4.1 Response Time	5
4.2 Failure Rate	5
4.3 Pod Scaling and Cost	6
4.4 P95 Latency	6
5 Statistical Analysis	6
6 Discussion	7
6.1 Key Findings	7
6.2 Proposed HPA Threshold Selection Framework	8
7 Limitations	8
8 Conclusion	8
References	9

Performance vs. Cost Efficiency in Kubernetes-Based Microservice Autoscaling

Afiya Farjana, Jannatul Ferdous, and Rumnaz Khan

Memorial University of Newfoundland, St. John's, NL, Canada

{afarjana, jferdous, rkhan}@mun.ca

Instructor: Dr. Kaiyang Liu COMP 6910 — Winter 2026

Abstract. Horizontal Pod Autoscaling (HPA) is the dominant autoscaling mechanism for Kubernetes microservices, yet limited empirical studies have quantified the cost-performance impact of CPU threshold selection under realistic workloads. This paper addresses the gap through 36 controlled experiments comparing three HPA configurations — 50% (aggressive), 70% (moderate), 85% (conservative) — across single-service and multi-service scopes under 100, 300, and 500 concurrent user loads on both Minikube and Google Kubernetes Engine (GKE). Results demonstrate that user load is the primary performance predictor ($r=0.988$ GKE, $r=0.800$ Minikube), pod count strongly predicts cost ($r=0.842$ GKE, $r=0.807$ Minikube), and infrastructure is the critical reliability factor — Minikube exhibited $\sim 24\%$ failure at 500 users while GKE maintained 0% failure. Alpha-beta network analysis ($\alpha=42.5$ ms, $\beta=1.67$ ms/KB) quantifies geographic latency as the primary contributor to GKE response time elevation, not compute inefficiency. An evidence-based, empirically validated HPA Threshold Selection Framework is proposed for practitioners.

Keywords: Kubernetes · HPA · Autoscaling · Microservices · Cloud Computing · Cost Optimization · GKE · Minikube

1 Introduction

Cloud-native applications increasingly rely on microservice architectures deployed on Kubernetes [5], where Horizontal Pod Autoscaling (HPA) provides reactive CPU-based scaling. The threshold that triggers scaling presents a fundamental dilemma: aggressive thresholds (50%) minimise response time but waste resources; conservative thresholds (85%) save cost but risk CPU saturation and degraded user experience. Despite this acknowledged trade-off, limited empirical studies have quantified the cost-performance impact of threshold selection under realistic workloads across both local and cloud environments [1]. Practitioners typically rely on default values or trial-and-error — a gap this study directly addresses.

We conduct 36 controlled experiments across two environments (Minikube, GKE), three thresholds (50%, 70%, 85%), two scaling scopes (single-service, multi-service), and three load levels (100, 300, 500 users), producing statistically validated findings and a practical decision framework.

Research Questions. **RQ1:** How does HPA CPU threshold affect response time and P95 latency under varying loads? **RQ2:** What is the quantitative cost-performance trade-off between 50% and 85% threshold strategies? **RQ3:** How does Minikube compare with GKE in reliability, performance, and cost under identical HPA configurations?

Contributions. (1) First empirical quantification of cost-performance trade-offs across three HPA CPU thresholds under realistic microservice workloads. (2) Cross-environment comparison revealing infrastructure capacity as the primary reliability determinant at high load. (3) Alpha-beta network model [6] isolating geographic latency from cloud application processing time. (4) An evidence-based, empirically validated HPA Threshold Selection Framework for practitioners.

2 Background and Related Work

2.1 Kubernetes HPA

Kubernetes HPA v2 [8] adjusts replica counts reactively using:

$$desiredReplicas = \left\lceil currentReplicas \times \frac{currentCPU\%}{targetCPU\%} \right\rceil \quad (1)$$

The ceiling function ensures demand is always satisfied. Scale-up cooldown is 15 s; scale-down requires 5 minutes of sustained under-utilisation. Lower thresholds trigger earlier and more aggressively, while higher thresholds delay scaling.

2.2 Alpha-Beta Network Model

The alpha-beta model [6] characterises network communication overhead as:

$$T(n) = \alpha + \beta \cdot n \quad (2)$$

where α (ms) is one-way base latency, β (ms/KB) is bandwidth coefficient, and n (KB) is message size.

2.3 Related Work

Rossi et al. [1] established threshold-based HPA as the production standard but did not quantify threshold-specific cost-performance trade-offs. The PMC Survey [2] reports 83% of container cloud spending is wasted on idle resources. Pandey et al. [3] proposed reinforcement learning autoscaling with improved efficiency but requiring ML complexity rarely adopted in production. Ahmad et al. [4] explored proactive scaling assuming abundant cloud resources — inapplicable in constrained environments. Burns et al. [5] introduced foundational container orchestration design principles establishing the architectural context for HPA. Our work uniquely provides empirical cost-performance data for simple threshold-based HPA without ML complexity, filling the practical guidance gap identified across all prior works.

Table 1: Experimental Design Variables

Factor	Levels
CPU Threshold	50% (aggr.), 70% (mod.), 85% (cons.)
User Load	100, 300, 500 concurrent users
Scope	Single-service (frontend); Multi-service (4 HPAs)
Environment	Minikube (local); GKE (cloud)

3 Methodology

3.1 Workload and Application

Google Online Boutique [9] — a 12-microservice e-commerce reference application — was deployed on Kubernetes via `kubectl apply -f kubernetes-manifests.yaml`. Locust 2.43 simulated product browsing (`GET /`), cart operations (`GET /cart`), and product viewing (`GET /product/{id}`). Analysis used Python on Google Colab.

3.2 Experimental Environments

Minikube (Local). 4 vCPU, 6 GB RAM (`-driver=docker`, Windows 11). Cost: mapped to GCP e2-standard-4 (\$0.134/hr) [7] $\Rightarrow$ \$0.015/pod/hr estimated.

GKE (Cloud). 3-node e2-standard-2 cluster (`us-central1-a`): 6 vCPU, 24 GB total [7]. Cost: $3 \times \$0.067/\text{hr} + \$0.10/\text{hr} = \$0.301/\text{hr} \Rightarrow \$0.01/\text{pod/hr}$. Actual billing: CA\$11.97 (free trial).

3.3 Experiment Design

3 thresholds $\times$ 3 loads $\times$ 2 scopes = 18 experiments/environment, 36 total (Table 1). Each experiment ran independently with full HPA deletion and pod reset between runs.

3.4 Cost Model

Since Minikube has no real billing and GKE uses node-based pricing, we normalise using a per-pod model [8]. **Step 1:** Minikube maps to e2-standard-4 at \$0.134/hr [7]; GKE 3-node cluster costs \$0.301/hr. **Step 2:** per-pod cost = total hourly cost $\div$ typical active pods (Minikube: $\$0.134/9 \approx \$0.015/\text{pod/hr}$; GKE: $\$0.301/30 \approx \$0.01/\text{pod/hr}$). **Step 3:**

$$\text{Cost} = \text{MaxPods} \times \frac{20}{60} \text{ hr} \times \text{CostPerPod} \quad (3)$$

A standardised 20-minute duration is used (pods stabilise in 4–5 min). This model simplifies node-based billing and does not account for idle charges, sustained-use discounts, or dynamic pricing — it provides a normalised basis for cross-threshold comparison only.

Table 2: Average Response Time (ms) — All 36 Experiments

		Minikube			GKE		
		100u	300u	500u	100u	300u	500u
50%	Single	102	406	936	299	1823	4185
50%	Multi	112	590	985	296	1675	4467
70%	Single	142	1529	936	301	1715	4070
70%	Multi	129	596	943	316	1939	4340
85%	Single	411	571	930	293	1707	4167
85%	Multi	123	677	918	296	1832	4370

3.5 Network Latency Characterisation

Ping analysis ($n=50$) from St. John’s, NL to GKE endpoint (`us-central1-a`, Iowa) yielded $RTT = 85$ ms (min 79, max 91, 0% loss), giving $\alpha = 42.5$ ms. HTTP measurements across 7–11 KB payloads yielded $\beta = 1.67$ ms/KB:

$$T(n) = 42.5 + 1.67 \cdot n \quad (\text{ms}) \quad (4)$$

Network overhead ($2\alpha = 85$ ms) accounts for 53.8% of the GKE–Minikube difference at 100 users and 45.7% at 300 users, confirming geographic distance — not compute inefficiency — drives GKE latency elevation.

3.6 HPA Validation

Eq. 1 validated empirically: at 70% threshold with 191% observed CPU, $\lceil 1 \times (191/70) \rceil = 3$ pods — matching every observed replica count in `kubectl get hpa -watch`.

4 Results

4.1 Response Time

Table 2 presents all 36 average response times. At 100 users Minikube performs well (102–411 ms). The critical divergence occurs at 300 users: delayed scaling causes sharp spikes at moderate thresholds — 70% Single reaches 1529 ms versus 406 ms for 50% Single, a $3.77\times$ gap. At 500 users all Minikube strategies converge (918–985 ms) at saturation. GKE values are consistently higher due to geographic network overhead (Eq. 4) — not compute inefficiency.

4.2 Failure Rate

Minikube exhibited $\sim 24\%$ failure at 500 users across all six threshold strategies (26.2% Single, 24.5% Multi), confirming the bottleneck is local resource ceiling, not HPA configuration. GKE maintained $\leq 0.1\%$ failure across all 18 experiments.

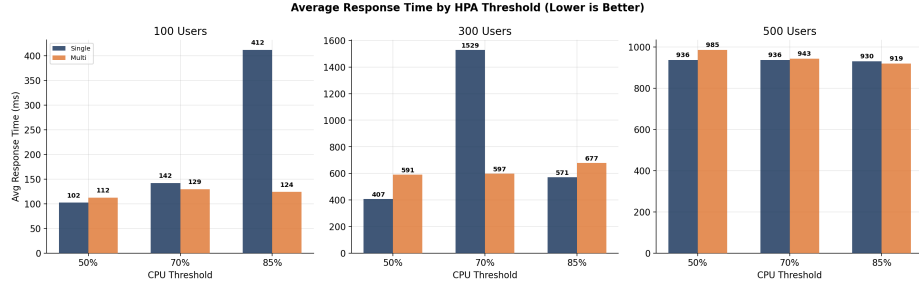


Fig. 1: Minikube: Avg Response Time by Threshold and Load.

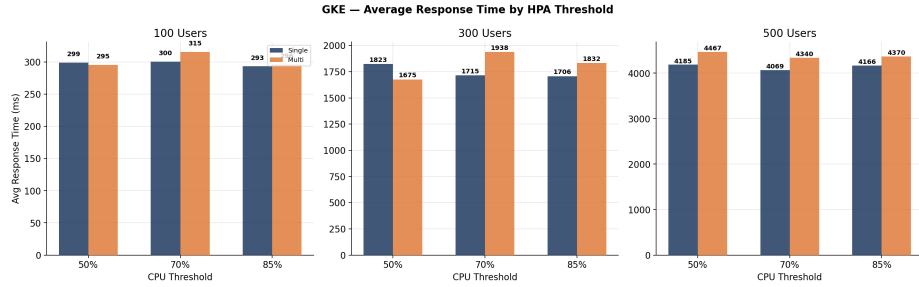


Fig. 2: GKE: Avg Response Time by Threshold and Load.

4.3 Pod Scaling and Cost

The 50% threshold deployed up to $3.3\times$ more pods than 85% (e.g. 50% Multi 300u: 10 pods vs. 85% Single 300u: 3 pods), directly multiplying cost. Multi-service experiments confirmed selective scaling: frontend and recommendationservice are CPU bottlenecks; cartservice and checkoutservice rarely exceeded 1 replica.

4.4 P95 Latency

P95 latency on Minikube reached 2000 ms at 500 users across all strategies, confirming saturation. At 100 users all strategies maintained P95 under 500 ms (270–490 ms). GKE P95 ranged from 610 ms to 6700 ms, elevated by geographic network overhead.

5 Statistical Analysis

User load strongly predicts response time ($r=0.988$ GKE, $r=0.800$ Minikube, $p<0.001$) and pod count predicts cost ($r=0.842$ GKE, $r=0.807$ Minikube, $p<0.001$). The non-significant threshold $\rightarrow$ RT ($p=0.69$) confirms threshold impact is weaker than workload impact — threshold tuning without load management is insufficient.

One-way ANOVA (threshold vs. RT, Minikube): $F=0.307$, $p=0.740$ — not significant. This indicates that threshold differences alone do not significantly affect response time when workload variance is considered, confirming load

Table 3: Failure Rate by Environment and Load

Env.	100u	300u	500u
Minikube	0%	0–3.2%	~24% (all 6 strategies)
GKE	0%	<0.1%	0% (all 6 strategies)

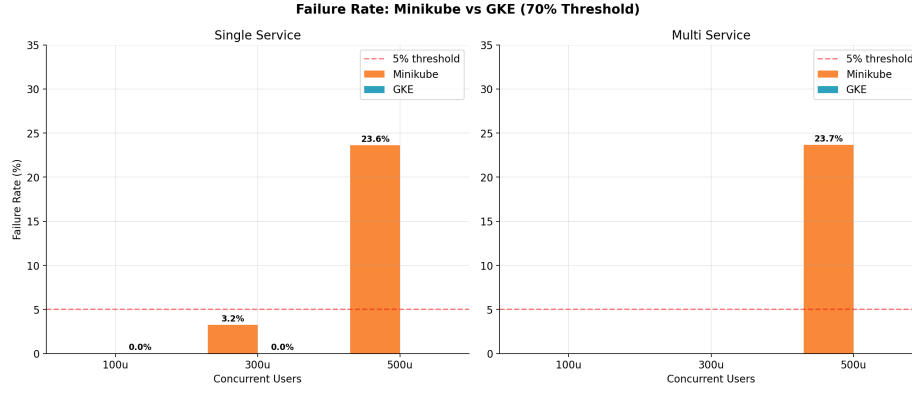


Fig. 3: Failure Rate: Minikube vs GKE (70% threshold).

management is a stronger lever than threshold tuning. t-test (Minikube vs. GKE): $t=-3.58$, $p=0.0008$ — environment type significantly affects latency.

6 Discussion

6.1 Key Findings

F1 — Load Dominates Performance. User load is the strongest predictor ($r=0.988$ GKE). Threshold alone is statistically insufficient (ANOVA $p=0.74$).

F2 — Pods Determine Cost. Pod count drives cost ($r=0.842$ GKE). 50% incurs up to $3.3\times$ higher cost than 85% at equivalent load.

F3 — Infrastructure Overrides Threshold. All six strategies failed at ~24% on Minikube at 500 users — infrastructure capacity, not threshold, determines reliability. GKE eliminated all failures.

F4 — Selective Multi-Service Scaling. Frontend and recommendationservice are CPU bottlenecks. Cartservice and checkoutservice rarely scaled — applying HPA uniformly wastes configuration effort.

F5 — Network Latency Explains GKE Elevation. Alpha-beta model ($\alpha=42.5$ ms) accounts for 53.8% of the GKE–Minikube difference at 100 users. GKE elevation is geographic in origin, not an infrastructure limitation.

F6 — 70% is the Optimal Balance. The 70% threshold achieves the best cost-performance positioning across both environments for workloads up to 300 concurrent users.

Core Insight. System behaviour is governed by three dominant factors: (1) workload determines latency, (2) pod scaling determines cost, and (3) infrastructure

Table 4: Average Estimated Cost per Experiment (USD, 20-min)

Threshold	Minikube avg	GKE avg
50% Aggressive	\$0.036	\$0.021
70% Moderate	\$0.028	\$0.018
85% Conservative	\$0.025	\$0.017
All 18 total	\$0.530	\$0.326

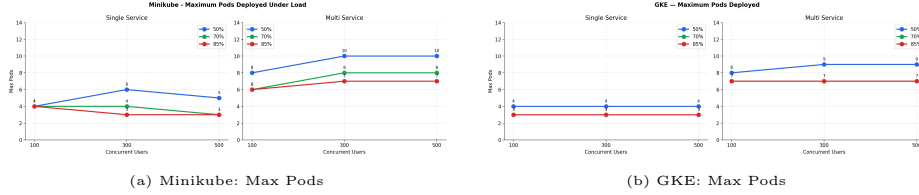


Fig. 4: Maximum Pods Deployed: Minikube vs GKE.

determines reliability. Tuning HPA thresholds in isolation — without considering workload and infrastructure — is insufficient for holistic system optimisation.

6.2 Proposed HPA Threshold Selection Framework

7 Limitations

Internal: Warmup effects observed; Locust shared host resources with Minikube. Only CPU-based autoscaling considered.

External: Results specific to Online Boutique HTTP workload; generalisation to I/O or memory-bound apps requires further study.

Construct: Minikube cost estimated (not billed); GKE uses list pricing without enterprise discounts.

Statistical: 18 data points per environment may be underpowered for detecting small effect sizes, particularly for ANOVA where within-group variance (load level) dominates between-group variance (threshold).

8 Conclusion

This study provides empirical quantification of cost-performance trade-offs across HPA CPU threshold configurations in a Kubernetes microservice environment. Through 36 controlled experiments on Minikube and GKE: (1) user load dominates response time ($r=0.988$ GKE, $r=0.800$ Minikube); (2) pod count drives cost ($r=0.842$ GKE); (3) infrastructure determines reliability — Minikube fails at 500 users while GKE achieves 0%; (4) 70% threshold provides the best balance up to 300 users. Alpha-beta analysis ($\alpha=42.5$ ms, $\beta=1.67$ ms/KB) validates GKE latency is geographic in origin. Tuning HPA thresholds without considering workload and infrastructure is insufficient — threshold selection must be part of a broader resource management strategy. The proposed framework provides actionable, statistically grounded guidance for practitioners.

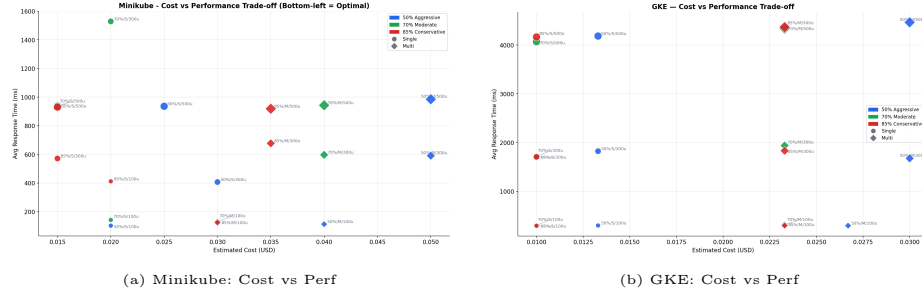


Fig. 5: Cost vs Performance Trade-off. Bottom-left = optimal.

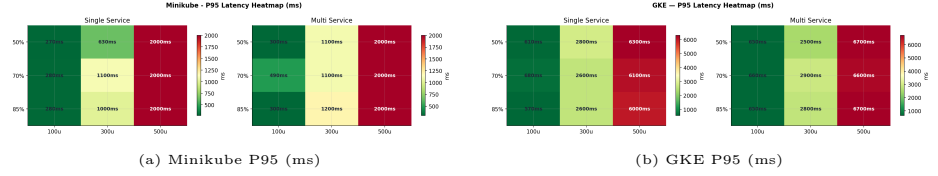


Fig. 6: P95 Latency Heatmap. Green = low; Red = high.

Future work includes GKE stress testing beyond 500 users, RL autoscaling comparison [3], multi-metric HPA (CPU + memory + RPS), and actual billing validation against the cost model.

References

- Rossi, F., Cardellini, V., Lo Presti, F., Nardelli, M.: Horizontal and vertical scaling of container-based applications. *Sensors* **20**(16), 4621 (2020)
- Survey on auto-scaling techniques for cloud computing. PMC 11398277 (2024)
- Pandey, V., Barolli, L., Ishida, T., Dantas, M.: Reinforcement learning-based autoscaling for cost and performance optimization in Kubernetes clusters. In: *3PGCIC 2025. LNDECT*, vol. 277, pp. 25–37. Springer (2025). https://doi.org/10.1007/978-3-032-10344-4_3
- Ahmad, H., Treude, C., Wagner, M., Szabo, C.: Towards resource-efficient reactive and proactive auto-scaling for microservice architectures. *J. Syst. Softw.* **225**, 112390 (2025). <https://doi.org/10.1016/j.jss.2025.112390>
- Burns, B., Grant, B., Oppenheimer, D., Brewer, E., Wilkes, J.: Borg, Omega, and Kubernetes. *Commun. ACM* **59**(5), 50–57 (2016). <https://doi.org/10.1145/2890784>
- Hockney, R.W.: The communication challenge for MPP: Intel Paragon and Meiko CS-2. *Parallel Comput.* **20**(3), 389–398 (1994). [https://doi.org/10.1016/0167-8191\(94\)90021-3](https://doi.org/10.1016/0167-8191(94)90021-3)
- Google Cloud: Compute Engine machine type families — E2 instances. <https://cloud.google.com/compute/docs/machine-resource> (2024)
- Kubernetes Project: Horizontal Pod Autoscaling — v2 API. <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/> (2024)
- Google Cloud: Online Boutique microservices demo. <https://github.com/GoogleCloudPlatform/microservices-demo>

Table 5: Pearson Correlation Results

Metric Pair	Mink. r	GKE r	Sig.
Load $\rightarrow$ Response Time	0.800	0.988	$p < 0.001$ ✓
Max Pods $\rightarrow$ Cost	0.807	0.842	$p < 0.001$ ✓
Threshold $\rightarrow$ Max Pods	-0.405	-0.260	$p > 0.05$
Threshold $\rightarrow$ RT	0.101	-0.001	$p = 0.69$

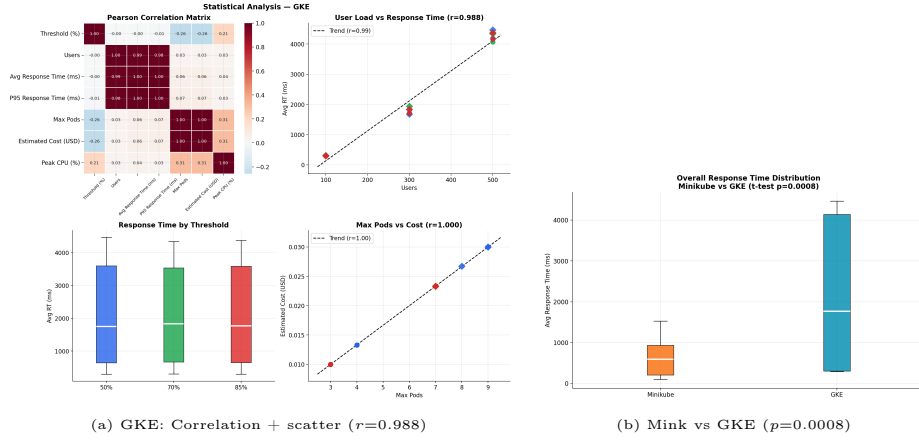


Fig. 7: Statistical analysis results.

Table 6: HPA Threshold Selection Framework (36 experiments)

Condition	Rec.	Evidence
Load $\leq 300u$, cost pri- ority	85% Conserv.	\$0.025 vs \$0.036 (50%); saves 31%
Load $\leq 300u$, perf. pri- ority	50% Aggress.	102ms vs 411ms at 100u; $3.7\times$
Load $\leq 300u$, balanced	70% Mod.★	Optimal scatter; both environments
Load $> 300u$ (any)	GKE + 70%	Minikube 24% fail; GKE 0%

★ Recommended default. Validated across 36 experiments.